class-75 12/08/2025

Spring ORM / Spring Hibernate

* ORM - Object Relational Mapping.
* DriverManagerDataSource.
  ↓
* Object pass to sessionFactory (Interface)
  ↓
  Class to implement sessionFactory is LocalSessionFactory.
  → It needs datasource bean. (we create bean to it)
* Transaction begin & Transaction commit will managed by Transaction Management with annotation called *** @Transactional
  → It will → begin transaction
  → commit transaction
  → close the transaction.
  ↔ it will roll back the data.
  → @Transactional will be placed upon created method.
  → It automatically handles the session, (it needs session factory obj)

Ex: Create Maven Project - SpringOrm

pom.xml

dependencies
- mysql-connector
- Spring context (6.2.9)
- Spring ORM (6.2.9)
- Hibernate core (version - 6.6.25) [org.hibernate.orm]

beans.xml (src.main.resources)

→ xsd bean

```
<bean id="dataSource" class="org.springframework.jdbc.core.DriverManagerDataSource">
  <property name="url" value="jdbc:mysql://localhost:3306/flmadu_java"></property>
  <property name="username" value="root"></property>
  <property name="password" value="Gayatrisrivikumar@2000">
</beans>
```

```
<bean id="session" class=org.springframework.orm.hibernate5.
                        LocalSessionFactoryBean" > </prop
<property name="dataSource" ref="dataSource" > </property>
<property name="hibernateProperties" >
</bean>

// to work @Transactional annotation,

<bean id="transactionManager" class="org.springframework.
                        orm.Hibernate5.HibernateTransactionManager" >
<property name="sessionFactory" ref="session" > </property>
</bean>

<property name="hibernateProperties" >
    <props>
        <prop key="hibernate.show-sql" > true </property>
        <prop key="hibernate.dialect" > org.hibernate.dialect.
                                MySQL8Dialect </prop>
        <prop key="hibernate.hbm2ddl.auto" > </prop>
                                none
    </props>
</property>
<property name="packagesToScan" value="com" > </property>
</bean>

<context:annotation-config> </context:annotation-config>
<context:component-scan base-package="com" > </context:component-scan>
<tx:annotation-driven/>
```

@EnableTransactionManagement.

Employee class

```
package com.model

@Entity
@Table(name="employees")
public Class Employee {
  @Id
  @GeneratedValue(strategy = GenerationType.IDENTITY)
  private int empId;

  private String email;
  private double sal;

public Employee() {

}
```

Constructor, Getter, setter & toString.

```
public Employee(String email, double sal) {

}
```

```
package com;

public class Test{

        PSVM {

    ApplicationContext container = new ClassPathXmlApplicationContext(
                                        "beans.xml");
//Sessionfactory
    sessionfactory = container.getBean("session", SessionFactory.class);
EmployeeDao employeeDao
    = container.getBean("employeeDao", EmployeeDao.class);
     Employee employee = employeeDao.getEmployee(1);
       Syso(employee);

//Save
     employeeDao.SaveEmployee(new Employee("bcd@gmail.com", 2340);

        ---x---

// Noxml
ApplicationContext container = new AnnotationConfigApplicationContext(
                                        config.class);
```

(Data Access Object)

```
package com.dao;
@Component @Transactional
public class EmployeeDao {  @Autowired
                    SessionFactory sessionFactory;
    public Employee getEmployee (int id) {

        Session session = getSession();
Employee
  employee = session.find ( Employee.class, id);
        return employee;

    }

    public session getSession() {
      return sessionfactory.getCurrentSession(); → [in place of
                                                    Opensession]
    }

//Save
    public void saveEmployee (Employee emp) {
        Session session = getsession();
        session.persist(emp);
    }
    // session.persist (new Employee ("gayi@gmail.com",9848);
    // if (true) {
        throw new RuntimeException();
    }
```

// with no beans.xml.

project : SpringNoXml

```
package com.config;

@Configuration
@ComponentScan("com")  @EnableTransactionManagement
public class Config {
  @Bean  //datasource
    public DataSource getDataSource() {
        return new DriverManagerDataSource("jdbc:mysql://
localhost:3306/.jbdt_adv_java", "root", "Gayatriravikumar@2000");
    }
    //localSessionFactory.
  @Bean Local
    public SessionFactory getSessionFactory() {
        LocalSessionFactoryBean localSessionFactoryBean = new
                LocalSessionFactoryBean();
        localSessionFactoryBean.setDataSource(dataSource());
        localSessionFactoryBean.setPackagesToScan("com");
        localSessionFactoryBean.setHibernateProperties(hibernateProperties());
        return localSessionFactoryBean;
    }
//hibernate properties
    public Properties hibernateProperties() {
        Properties props = new Properties();
        props.put("hibernate.show-sql", "true");
        props.put("hibernate.dialect", "true");
                                  org.hibernate.dialect.MySQL8Dialect
        props.put(
        return props;
    }
```

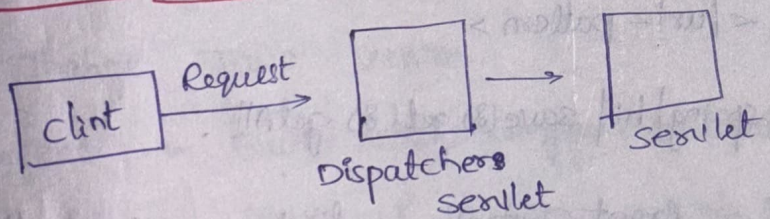
```
@Bean                                          SessionFactory sessionFactory
public HibernateTransactionManager transactionManager() {
    HibernateTransactionManager tManager = new HibernateTransaction
                                                    Manager();
    tmanager.setSessionFactory(SessionFactory);
    return tManager;
}
}
```

13/08/2025

Spring MVC

class-76

* MVC - Model View Controller (design pattern)
* Models - kind of pojo classes / data.
* Views - either HTML (or) JSP pages.
* Controller - kind of servlet which hander http request.
* **FRONT CONTROLLER DESIGN PATTERN.**



Dispatcher Servlet:

* Whenever client sends a request, it takes the request & (must) to the servlet redirect to
* It handler all multiple servlets.
* We have to url pattern → @Controller (like /hi)
  → ~~localhost~~ Ex: http://localhost:8080/Spring/hi.

Ex:

```
@Controller
public class EmployeeController {
    @RequestMapping("/get") → APIs (or) End points.
    public void getEmployee() {
```

```
@RequestMapping("/save")
public void saveEmployee() {

}

@RequestMapping("/getAll")
public void getAllEmployees() {

}
```

web.xml

```
<servlet>
    <servlet-name>ds</servlet-name>
    <servlet-class>org.spring.DispatcherServlet</servlet-class>
</servlet>
<url-pattern>hi</url-pattern>
```

⇒ http://localhost:8080/spring/hi/save(8)/get(8)/getAll.

* **DispatcherServlet** is a front controller which receives the request coming from the client & redirects them to corresponding controller.

HTTP Methods

* 1. GET – to return the data – @GetMapping (by server to client)
2. POST → to save when client sends some data, to be saved at server.
3. PUT → when client sends some modified data on existing data (update). [full update]
4. DELETE → when client asks to delete some data from server.
5. PATCH → It is also used for update, i.e, half update. (&) partial update.

* Status code : by server

↓

It sends by server (8) we can send them manually.

1. 1xx - infomational status code.
2. 2xx - success status code (Ex: 201 (Created) (post), 202, 204 (No content) (delete), ~~209~~, 200 (OK) (Get), ~~Conflict~~ (delete)
3. 3xx - Redirection
4. 4xx - client side error (400 (Bad Request), 401 (unauthorised) No access, 403 (Forbidden) (wrong login credentials), 404 (Not Found), 405 (Method Not Allowed), 409 - (conflict)
5. 5xx - server side error

→ 500 - Internal Server Error

→ 502 - Bad Gateway.

Class-77 14/08/2025

* Project - Maven project → Next → Filter (archetype webapp)

↓

given by maven-archetype-webapp(1.5) (org.apache.maven.archetype)

→ Next → GroupId ArtifactId → Finish.

* 1. Change Java version.

⇒ Project → Build path → Configure Buildpath → JRE System libraries - java

→ Edit → Java-17 → Apply → Apply & Close.

2. change web.xml ~~xsd~~ xsd - update.

3. Buildpath → configure buildpath → Project facets → change versions of Dynamic web module - 6.0 & Java - 17 → Apply → Apply & close.

4. Target Runtimes (configure buildpath) - select tomcat - 10

web.xml

```
<servlet>
<servlet-name>flm</servlet-name>
<servlet-class>org.springframework.web.servlet.DispatcherServlet
</servlet-Class>

</servlet

<servlet-mapping>
<servlet-name>flm</servlet-name>
<url-pattern>/flm/*</url-pattern>
```

</servlet-mapping>

</web-app>

pom.xml

→ add spring-context & spring webmvc dependencies.

* If servlet name is flm, we have to create xml file with name flm-servlet.xml → this is for creating beans. (beans.xml)

* xml file ⇒ flm-servlet.xml

(WEB-INF) <context:annotation-config>

<context:component scan base-package="com"/>

* Java Resources → src/main/java → New → Class

Class:

```
package com.controller;

@Controller (this inform to server that this is controller class & creats a bean also)
@RequestMapping("/fet")
public class HomeController {
    @RequestMapping("/hi") → endpoint
    @ResponseBody
    public String hi() {
        //return "Hii";
        return "home.jsp"; ⇒ It gives data.
    }

    @RequestMapping("/home")
    public String hi2() {
        return "home"; ⇒ It gives home.jsp file.
    }
    @RequestMapping("/data")
    public String home(Model model)
        return "home";
        model.addAttribute("name", "Gayatri");
        return "home";
```

(Controller internally has @Component)

["http://localhost:8080/SpringMVC/flm/hi"]

Class (org.springframework.ui.Model)

(* One Controller can have multiple methods)

```
package com.Controller;

@Controller
public class ByeController{
    @RequestMapping("/bye")
    @ResponseBody
    public String bye(){
        return "Bye";
    }
    @RequestMapping("/signup")   (to send data to ~~servlet~~ next JSP)
    public String signup(Model model, HttpServletRequest http){   (to read data from coming JSP)
        String email = (String) http.getParameter("email");
        String password = (String) http.getParameter("password");   (@ModelAttribute)
        model.addAttribute("email", email);
        model.addAttribute("password", password);
        return "data";
    }
```

JSP Page

webapp → New → JSP File → home.jsp

```
<body>
<h1> Hi Welcome !! ${name} !! </h1>
                    (to get name)
<form action="signup" method="post">
Email: <input type="text" name="email">
password: <input type="password" name="password">
<input type="submit" value="Submit">
</form>
</body>
```

* Web-INF → New → Folder → Views
→ put this home.jsp file into views.

* **Internal View Resolver:**

→ It takes two arguments to read data in views folder.

1. prefix → /WEB-INF/views/
2. suffix → .jsp

* Set this prefix & suffix in flm-servlet.xml.

```
<bean id="iirs" class=org.springframework.web.servlet.view.
                    InternalResourceViewResolver"></bean>
<property name="prefix" value="/WEB-INF/views/"></property>
<property name="suffix" value=".jsp"></property>
```

**
*

* @Controller + @ResponseBody → resolves data (@RestController)

* @Controller → Resolves views.

Data.jsp:

```
<body>
<h1> Hi your email is ${email} your password is
        ${password} </h1>
</body>
```

* http://localhost:8080/SpringMVC/flm/fet/hi ← If we use @RequestMapping in class level, the URL is

Class-78 18/08/2025

* Project with Noxml.

Maven Project – SpringMVCNOXML

To remove xml:

```
package com.config;

@Configuration
public class
@ComponentScan(basePackages = "com")
@EnableWebMvc
public class Config {

    @Bean
    public InternalResourceViewResolver internalResourceViewResolver(){

    InternalResourceViewResolver irvr = new InternalResourceViewResolver();
     irvr.setPrefix("/WEB-INF/views/");
     irvr.setSuffix(".jsp");
      return irvr;

    }
// to initialize dispatcherservlet, we have class called
AbstractAnnotationConfigDispatcherServletInitializer.
```

// to initialize Dispatcher Servlet,

```
package com.config;
public class DispatcherServletConfig extends AbstractAnnotationConfig
        DispatcherServletInitializer {

  @Override
    protected Class<?>[] getRootConfigClasses() {
     return null;

    }



  @Override
    protected class<?>[] getServletConfigClasses() {
     class[] configclasses = {Config.class};

      return configClasses;

    }
```

```
@Override
protected String[] getServletMappings() {
        String[] mappings = {"/flm/*"};
        return mappings;
    }
}
```

* Views folder (WEB-INF)

1. home.jsp

```
<body>
<h1> welcome to FLM!!! </h1>
</body>
```

Controller class:

```
package com.controller;

@Controller

public class HomeController {

    @RequestMapping("/home")

    public String home() {

        return "home";


    }
    @RequestMapping("/home2").
    @ResponseBody
    public String home2() {

     return "home";

    }

}
```